COMP3888 Music Evidence Evaluation Program Technical Manual

V3 architecture scoring and optional audio observations

Document field	Current value
Program version	0.7.0 with optional report-only audio integration
Document revision	18 October 2026
Applicable directory	program/
Primary audience	COMP3888 developers, assessors, and technical collaborators
Current verification	189 tests passed with no failures or skips
Branch baseline	feature/dynamic-weighting-with-cap at c6d36fd

This manual describes the V3 evidence evaluator on feature/dynamic-weighting-with-cap, based on commit c6d36fd, with report-only audio integration verified on 18 October 2026. The core validates the declared graph, binds files, runs parsers and checks, and reports four separate axes. Integrity uses three configured evidence layers and a severe-contradiction cap. The legacy signal and exact-reconstruction passes are disabled. When explicitly enabled, independent audfprint, SpectroMap and librosa workers append content and structure observations after the core scoring response is retained. They never alter scores, confidence, withholding, caps or submitted relationships.

The program does not produce an overall score, an authorship decision, an accepted or rejected outcome, or a human-versus-AI classification. Its output describes evidence coverage, internal consistency, available attestation, and disclosure coverage.

Contents

1 **Program Purpose and Workflow**

2 Supported Artefact Types

3 Supported Relationship Types

4 Supported Physical File Types

5 Passes and Checks

6 Four Output Axes

7 Program File Responsibilities

8 Outstanding Work

Terminology follows the codebase: an artefact is a declared evidence node, a relationship is a directed edge from a source node to a target node, and a pass is a bounded check that returns observations, detailed checks, findings, or an axis assessment.

1 Program Purpose and Workflow

The program receives a submitter-declared evidence graph through Native input or a CSEC request or public bundle. It validates structure, adapts agreed values, binds files by SHA-256, runs parsers and assessments, and validates the response. The optional local audio flag adds a separate diagnostic stage after core scoring; it does not extend the public request schema.

1 1 End to End Workflow

Stage	Operation	Result
1	Read request	Limit JSON to 2 MiB and reject duplicate object keys or non-finite numbers.
2	Validate structure	Apply the Native or CSEC JSON schema before semantic processing.
3	Adapt input	Use the Native chain directly or map agreed CSEC WAV and draft-of.audio values.
4	Normalise graph	Quarantine isolatable unknown values; retain supported artefacts and relationships.
5	Check graph safety	Enforce node limits, unique hashes, a declared final node, valid references, and structural connectivity.
6	Bind objects	Check the final object first and stop if it cannot be confirmed; then resolve, size-check and hash the other objects, isolating any that fail.
7	Run pipeline	Execute artefact and relationship passes with per-object result caching.
8	Parse declared types	Run WAV, MIDI, RIN, ERN, C2PA, CompSheet, or CueSheet logic only for the declared type.
9	Assess retained evidence	Calculate file, declaration, CrossEvidence, attestation, disclosure and workflow results; legacy signal checks stay disabled.
10	Retain core response	Validate and retain axes, checks, findings, coverage and validation before optional diagnostics.
11	Optional audio observations	With the explicit local flag, verify snapshots and run isolated providers within the remaining caller time.
12	Append and validate	Append AUDIO_* findings and diagnostic arrays; retain core scores and original findings even if diagnostics fail.

1 2 Unknown and Invalid Input Handling

The normalisation boundary separates local semantic problems from request-wide structural problems. Unknown declarations that can be isolated are removed from the evaluation graph and reported. A known non-final WAV that cannot be parsed remains visible in the graph, but only WAV-dependent checks become unavailable. A structural problem, an unreadable final artefact, or an unsafe graph stops the request because the program cannot form a reliable evaluation root. Input errors are reported and never corrected: a problem with the final artefact stops the request and only that problem is reported, while a problem with another artefact or relationship makes only the checks that depend on it unavailable.

Input condition	Current action	Reported state
Unknown non-final artefact type	Quarantine the artefact and relationships that depend on it; continue checking retained files.	execution_status succeeded, coverage complete false

Input condition	Current action	Reported state
Unknown relationship type	Skip only that relationship; do not select a similar registered relationship.	unknown_inputs; finding code: UNKNOWN_RELATIONSHIP_ TYPE_SKIPPED
Known relationship with incompatible endpoint roles	Skip only that edge and continue with the remaining graph.	RELATIONSHIP_ENDPOINTS_SKIPPED
Known non-final WAV cannot be parsed	Keep the node and declaration, record parser_failed, and skip only checks that require decoded WAV data.	execution_status succeeded; WAV_PARSER_FAILED; dependent checks unavailable
Final artefact file, hash, mapping, path or size cannot be confirmed	Stop at once; report only the final artefact's problem and do not check other objects.	execution_status error (unsupported for the size limit or an unsupported container)
Non-final file with a SHA-256 mismatch, size-limit excess or path outside the root	Do not bind the file; skip only checks that need it; workflow rules that rely on it are not_assessed.	OBJECT_HASH_MISMATCH, OBJECT_SKIPPED_SIZE_LIMIT or OBJECT_PATH_INVALID
Non-final upper-case hash or suffix, or file name not matching the hash	Do not correct it; do not bind the file; skip only checks that need it.	INVALID_HASH_FORMAT or INVALID_OBJECT_PATH
Unknown or misspelled attribute field	Do not correct it; skip the checks that use that declaration (a misspelled relationship field disables that relationship's content checks).	UNKNOWN_FIELD with location and suggested spelling
Unknown final artefact type	Stop because no supported evaluation root can be formed.	execution_status unsupported
Dangling hash, duplicate node, malformed schema, or unrelated supported node	Stop because the graph structure is unsafe or ambiguous.	execution_status error

When any artefact or relationship is skipped, or an input error makes checks unavailable (finding SCORE_AFFECTED_BY_INPUT_ERRORS), every otherwise available axis is downgraded to partial. The value remains the result for the retained evidence only. The coverage object states the submitted, evaluated, and skipped counts, so omitted evidence cannot appear to be a complete assessment.

1 3 System Boundaries

- The program does not select an artefact type from a filename, extension, parser result, or media content.
- The program does not correct malformed input such as upper-case hashes or suffixes or misspelled fields; it reports each problem with its location and a suggested fix.
- The program does not create a relationship from audio similarity, C2PA ingredients, DDEX content, or graph position.
- A parser observation may verify or contradict a declaration, but it cannot replace the declaration.
- Completeness measures expected evidence coverage. It does not prove that a production event occurred.
- The program does not calculate an overall score or make an acceptance, rejection, or authorship decision.

1 4 Design Principles

These principles govern declarations, parsers, scoring and optional audio observations. Estimated audio correspondences remain observations with an explicit source, parameter set and limitation; they do not become graph claims or scoreable checks.

Principle	Implementation	Boundary
Declared semantics are authoritative	Submitter types, direction, workflow and parameters drive evaluation.	Estimated audio locations never create or correct declarations.
Evidence layers stay separate	Four axes; three configured Integrity layers; C2PA and audio remain separate.	No overall score, authorship decision or proof of historical direction.
Conservative diagnostic states	Uncalibrated completed analyses are inconclusive; analyses that cannot run are unavailable.	A candidate match is not verification; a non-match is not a contradiction.
Validated isolated execution	Existing file/path/schema checks and caller timeout apply. Providers run in separate processes after core scoring.	No added development caps; failed diagnostics preserve core results.
Partial evaluation is visible	Coverage, unknown_inputs, statuses and reason codes identify omissions.	Skipped inputs and partial analysis are not silently treated as assessed.
Reproducibility	Pinned providers, parameter hashes, source records and engineering fixtures identify the run.	Synthetic tests do not replace real-session accuracy calibration.

2 Supported Artefact Types

An artefact type is a semantic role chosen by the submitter. It is distinct from the physical file format and from observations extracted by a parser. The type controls which attributes, relationships, parsers, and workflow rules may apply.

2 1 Audio Artefact Types

Type identifier	Meaning	Current use
audio	Generic audio	Used by the CSEC WAV adapter without inventing a production role.
audio/raw-take	Raw performance take	An original recorded or rendered take before comping.
audio/raw-track	Raw source track	A source track before stem, mix, or master production.
audio/edited-take	Edited or comped take	A take created by declared editing or comping.
audio/stem	Grouped stem	An export grouped by instrument, voice, or production function.
audio/mix	Mix	A result formed from multiple declared audio sources.
audio/master	Master	A mastered or final-delivery audio result.
audio/sample	Sample	An excerpted, looped, or reused audio segment.
audio/reference	Reference audio	Audio supplied for comparison or production guidance.
audio/ai-source	Declared AI source	Audio that the submitter explicitly identifies as AI-generated.
audio/unclassified	Unclassified audio	Audio whose submitter does not assign a more specific role.

2 2 Non Audio Artefact Types

Type identifier	Category	Current handling
project/midi	Project	Bounded Standard MIDI File parser.
project/daw-session	Project	May be declared; no generic proprietary session parser.
project/automation	Project	May be declared and checked through relationships and attributes.
metadata/ddex-rin	Metadata	RIN 2.1 parsing and offline XSD validation.
metadata/ddex-ern	Metadata	ERN 4.3 parsing and offline XSD validation.
provenance/c2pa	Provenance	Optional c2pa-python reading and validation.
text/cue-sheet	Document	JSON, CSV, delimited text, and text-extractable PDF.
text/comp-sheet	Document	JSON, CSV, delimited text, and text-extractable PDF.
text/session-log	Document	Presence evidence; no dedicated semantic parser.
text/license-clearance	Document	Presence evidence for licence and clearance records.
text/generation-record	Document	Evidence for declared AI or generation processes.

3 Supported Relationship Types

Relationships are directed from a source artefact to a target artefact. In JSON, the edge is stored in the target artefact's evidence array, and hash identifies the source. A relationship is a submitter declaration. Registration and endpoint validation do not prove that the stated transformation occurred.

Relationship	Allowed source	Allowed target	Meaning
derived_from	Any registered type	Any registered type	General declared derivation.
edited_from	raw-take, raw-track, edited-take	edited-take	Editing produced the target take.
excerpted_from	Any registered type	sample	The sample was excerpted from the source.
comped_from	raw-take	edited-take	The target combines selected takes.
stemmed_from	raw-track, edited-take, sample	stem	The target is a grouped stem.
mixed_from	stem, raw-track, edited-take, sample	mix	The target mix uses the declared sources.
mastered_from	mix	master	The target master derives from the mix.
rendered_from	MIDI, DAW session, automation	Specific audio role	The target audio was rendered from a project source.
input_to	Any registered type	Any registered type	The source was an input to the target process.
draft-of.audio	audio	audio	Generic CSEC-compatible audio draft relationship.

3 1 Relationship Attributes

Attribute group	Fields	Purpose
Declaration source	assertion_origin; confirmed_by_submitter	Records where the claim originated and whether the submitter confirmed it.
Explanation	reference_pointer; claim_rationale; transformation_description	Stores a reference, rationale, or description without treating it as proof.
Analysis scope	derivation_scope: linear_edit, linear_stem, linear_mix, or master_similarity	Retained declaration vocabulary. These values do not activate deprecated reconstruction. Optional diagnostics use their own explicit local policy.
Time range	source_start_seconds; source_end_seconds; target_start_seconds; target_end_seconds	Core parameter checks compare ordering and bounds. Optional range analysis requires all four endpoints and reports original-file times.
Technical parameters	gain; fade_in_seconds; fade_out_seconds; input and output sample rate, bit depth, and format	Core technical checks use available measured fields. Gains and fades do not activate exact signal reconstruction.

4 Supported Physical File Types

The objects map binds a declared SHA-256 digest to a bundle-relative path. A suffix does not select a semantic role or prove an audio format. The core resolves paths inside the root, checks the final object first, applies

existing size limits, hashes and binds valid bytes, and dispatches the declared parser. Optional diagnostics independently rehash temporary WAV snapshots before reading samples; a changed snapshot input is rejected for that diagnostic.

Format	Suffix	Status	Checks and limits
WAV	.wav	Implemented	Streaming RIFF/WAVE inspection; integer PCM (8/16/24/32-bit) and IEEE float (32/64-bit), including matching WAVE_FORMAT_EXTENSIBLE SubFormats; channels, rate, bit depth, data length, and declared technical fields. Trailing bytes and unfinalised sizes are warnings; RF64/BW64 and compressed codecs are unsupported. A non-final parser failure is isolated; the final remains strict.
MIDI	.mid .midi	Implemented	Tracks, notes, tempo, time signature, controllers, and calculable duration.
DDEX XML	.xml	Implemented	RIN 2.1 and ERN 4.3; offline XSD; DTD and entity declarations rejected.
Sheet JSON	.json	Implemented	Strict or normalised CompSheet and CueSheet input.
Sheet CSV	.csv	Implemented	Field aliases and tabular records.
Sheet text	.txt	Implemented	Delimited text and bounded normalisation.
Sheet PDF	.pdf	Partial	Text extraction only; no OCR for scanned documents.
C2PA	embedded or .c2pa	Optional	Requires c2pa-python; distinguishes SDK, reader, manifest, and validation states. Embedded and standalone records are bound to graph assets before cross-checking.
DAW archive	.zip .als .logicx .ptx	Packaging only	Schema permits packaging, but proprietary session content is not parsed.
PNG and MP4	.png .mp4	CSEC audio target unsupported	A final unsupported media type returns unsupported; a non-final item may be quarantined.

Existing core limits remain 512 MiB per object and 1 GiB per request. WAV headers and chunks are inspected incrementally. The caller options.timeout_seconds controls execution; 60 seconds is a default, not an evaluator ceiling. The proposed diagnostic caps for runtime, relation count, file duration, active PCM and feature frames were not added. Diagnostics use the remaining caller time and never silently inspect only the opening of a long file. Invalid non-final inputs are isolated; an invalid final input stops core evaluation. A non-final change during core evaluation permits one retry with the file isolated. A later diagnostic snapshot mismatch makes only that diagnostic unavailable and preserves the completed core response.

5 Passes and Checks

The core Pipeline executes WavMetadataPass per audio artefact and RelationshipIntegrityPass per retained edge. BoundFileIntegrityPass reuses their cached technical checks. Type-specific parsing feeds CrossEvidence, C2PA, AI disclosure and workflow completeness. assess_axes emits disabled physical-similarity placeholders and does not execute legacy DSP or exact reconstruction. With the audio flag, the parent process subsequently prepares validated tasks and snapshots and runs isolated optional providers. Their observations are appended after scoring and never passed back through the scored Boolean checks.

5 1 Active Passes

Pass	Scope	Checks	Output and cache
WavMetadataPass	Audio artefact	Streaming RIFF/WAVE structure for integer PCM and IEEE float, including EXTENSIBLE SubFormats; RIFF warnings; WAV metadata; and standardised technical checks; non-final failures become parser_failed.	Measured WAV observations and checks cached per artefact; the core does not scan samples for disabled signal comparisons.
RelationshipIntegrity Pass	Relationship EvidencePass	Declared time ranges, duration bounds, sample rate, and bit depth compared with cached endpoint observations.	Observation, checks, and reasoning; cached per edge.
BoundFileIntegrity Pass	Graph aggregation	Object binding, parser failures, and cached WavMetadataPass checks; unavailable checks are excluded from the numeric denominator.	Value, checks, and findings; WAV comparisons are not repeated.
Expanded CrossEvidencePass	Graph	RIN, ERN, MIDI, sheets, embedded and standalone C2PA ingredients, contributor, identifier, and AI claim consistency; bad optional field types are isolated per check.	Value, checks, findings; graph-level cache pending.
C2PAAttestationPass	Final artefact and record scope	Final-bound manifest, signature, validation and credential trust; supporting records separate.	Attestation axis, validation and C2PA records.
AIDisclosurePass	Retained audio nodes	Coverage of explicit creation_method declarations.	AI disclosure axis and findings.
Workflow CompletenessPass	Selected workflow	Required, conditional and optional expectations.	Completeness axis and rule results.
Optional diagnostic runner	Parent after core response	Declared audio pairs, verified snapshots and isolated audfprint, SpectroMap and librosa workers.	Non-scoring AUDIO_* findings and separate diagnostic arrays.

5 2 Type Specific Parsing

parse_bound_artefacts is currently an orchestration function rather than a registered Pass. It selects a parser from the declared artefact type, records parser status and data, and passes failures to later assessments. WAV parsing reuses WavMetadataPass status; unsupported non-final WAVs are not decoded again. It does not infer a type from the file.

Declared type	Parser action
audio types	Reuse WavMetadataPass observations and status; attempt embedded C2PA reading. A non-final parser failure is not retried.
project/midi	Run the bounded MIDI parser.
text/comp-sheet	Run the CompSheet parser.
text/cue-sheet	Run the CueSheet parser.
metadata/ddex-rin	Run the RIN parser and record XSD status.
metadata/ddex-ern	Run the ERN parser and record XSD status.
provenance/c2pa	Run the standalone C2PA parser, bind the record to its adjacent graph asset, and include it in ingredient cross-checks.

Declared type	Parser action
other registered types	Record not_assessed; do not select another parser.

5 3 Pass Result Reuse and Dependency Cycles

Artefact and Evidence objects cache results by Pass class. RelationshipIntegrityPass and file-integrity aggregation reuse WavMetadataPass results. Verified hashes are reused for core binding; source_revision is cached once per process and includes the added audio sources and dependency records. Core WAV descriptors remain reusable. The optional runner caches verified snapshots for the current invocation; legacy PCM-window and low-rate proxy helpers are not active scoring dependencies. Each Artefact or Evidence object maintains a Pass call stack and raises CircularEvaluationException for recursive evaluation.

Registered Pass caching and diagnostic snapshots are request-scoped. Shared caching of type-specific parsers and graph indexes remains future work. No diagnostic result is reused across requests without explicit identity and parameter validation.

5 4 Checks Outside the Main Execution Path

Component	Current status
CrossEvidencePass	Earlier implementation retained for reference; assess_axes uses ExpandedCrossEvidencePass.
vendor builtin C2PAPass	Upstream draft; the target uses the legacy parser plus C2PAAttestationPass.
ArtefactPass and EvidencePass	Abstract interfaces; they do not produce results on their own.
Legacy derivation and physical passes	Disabled in the assessment path. Exact reconstruction and all eight former physical diagnostics are retained as historical code only.

5 5 Optional Audio Providers

The --audio-analysis flag enables all three providers in the local default policy. --audio-analysis-policy selects a local policy file and can enable a subset. Neither flag changes the Native or CSEC request schema. Without a flag, optional libraries are not loaded and no diagnostic worker runs.

Provider	Implemented observation
audfprint	Pinned upstream landmark extraction, deterministic local offset voting, repeated-source candidates and supported windows.
SpectroMap	Upstream spectral peak detection on fixed 4-second windows, so peak density does not depend on file length, with an exact peak-pair matcher and offset voting. nfft and noverlap define the frequency/time grid.
librosa	Chroma, MFCC and energy features, time aggregation, candidate boundaries and sparse recurrence within each file; cross-file similarity is reported only as summary statistics because it did not separate related from unrelated real recordings. A separate report-only music_semantics provider compares key, tempo, beat chords and 16-beat phrase form across versions, with transposition, tempo ratio, harmonic similarity against a beat-shuffled baseline, chord-sequence similarity and phrase order.

The providers analyse only retained, declared audio-to-audio relationships with usable core-bound files. CompSheet tasks require matching source/target selections. A selection without target_end_seconds copies its declared source span and is flagged range_source derived_from_declared_source_duration; other supplied ranges require all four endpoints. Estimates are returned in original-file seconds and never replace edge parameters. Silence, channel cancellation and insufficient signal are diagnostic limitations, not contradictions.

5 6 Execution and Dependencies

The parent retains the completed core response, verifies temporary WAV snapshots and launches one isolated worker per task, which runs every provider in turn, decodes each file range once and streams one result per provider, so results finished before a timeout or crash are kept. Snapshots are rehashed and reused during that invocation. Provider exceptions, missing optional packages, invalid policies and caller-time exhaustion append unavailable observations without changing the retained axes or original core findings. A crashed provider's diagnostic includes the tail of its stderr; a protocol-breaking result or a worker that cannot start affects only its own task and provider.

Run the following commands from program/. The vendored audfprint extractor uses decoded arrays, so this integration does not require FFmpeg. requirements-audio-lock.txt records the tested environment; music_target/audio_diagnostics/vendor/UPSTREAM.json records sources, commits and licences.

```
python3 -m pip install -r requirements-audio.txt
python3 -B make_audio_diagnostic_example.py
python3 -B run.py --native examples/audio-diagnostics/request.json
  --root examples/audio-diagnostics --audio-analysis
  --output examples/audio-diagnostics/response.json
```

The last three lines form one command; join them with spaces when entering it. Diagnostics consume only the remaining options.timeout_seconds budget. The proposed 30-second diagnostic, 10-second provider, 16-relation, 600-second file, 64-MiB active-PCM and 1500-frame caps were not implemented. Existing file checks remain active, and a long file is not silently truncated.

5 7 Diagnostic Results

AUDIO_FINGERPRINT_OBSERVATION, AUDIO_STRUCTURE_OBSERVATION and AUDIO_DIAGNOSTIC_UNAVAILABLE findings link to detailed object arrays through diagnostic_id and detail_key. Details are in audio_fingerprint, audio_structure and audio_diagnostic_summary under assessment_checks. Each record preserves provider/version, policy hash, parameters, ranges, candidates, metrics and limitations.

Every diagnostic fixes scoring_eligible to false, integrity_contribution to null and content_match_is_provenance to false. Current candidate thresholds are uncalibrated: completed analysis is inconclusive, even when matches exist; an analysis that cannot run is unavailable. The measurements are never averaged into a provenance score. A report-only fingerprint_agreement row lists offsets where audfprint and SpectroMap both voted, including sub-threshold votes, with a Poisson chance estimate; both read the same audio, so agreement is not independent evidence. A graph without audio-to-audio relationships yields one AUDIO_DIAGNOSTIC_NOT_APPLICABLE finding. The old physical_similarity placeholders continue to identify the disabled legacy layer.

5 8 Verification and Interpretation

Verification on 18 October 2026 passed 127 core tests, 34 diagnostic tests and 9 production-document tests, with no failures or skips. Real installed backends completed the generated repeated-excerpt example. audfprint found offsets near 2.00 and 10.01 seconds; SpectroMap found offsets near 2.00 and 9.996 seconds; librosa returned structure, repetitions and feature correspondences. Enabled and disabled runs retained identical four-axis values and original findings. A 20-second excerpt in a 3-minute source is a regression test: whole-file SpectroMap peak search aligned about 3% of the excerpt's peak pairs, while the windowed search aligns about 62%. These are engineering results; real-session accuracy and thresholds still need labelled data.

```
REQUIRE_AUDIO_BACKENDS=1 python3 -B -m unittest discover -s tests -v
```

6 Four Output Axes

The four axes are independent. A value lies between 0 and 1, but zero and one have a different meaning for each axis. The values must not be added or interpreted as a probability that the submitter is the creator.

6 1 Common Axis Attributes

Attribute	Allowed value	Meaning
availability	available, partial, unavailable	Whether the axis can be interpreted from the evidence examined.
value	0 to 1 or null	The axis result. null means that no safe numeric result is available.
confidence	0 to 1 or null	Coverage or evidence availability, not a probability error bound.
reasoning	string or null	Checks included, scope limits, and reasons for missing assessment.

6 2 Completeness

Completeness measures how much of the evidence expected by the submitter-selected workflow is present and assessable. It does not prove that the submitted claims are true.

The evaluator applies common rules, workflow required rules, and conditional rules whose triggers are true. Optional rules are reported but excluded from the denominator. The value is the weight of satisfied assessed rules divided by the weight of all assessed required and conditional rules. Confidence is the number of assessed applicable rules divided by all applicable rules.

If the final artefact type is not allowed by the workflow, an existing completeness value is set to zero. If no workflow is supplied through the compatibility path, completeness uses bound retained objects divided by retained nodes. Input or coverage issues can downgrade availability to partial. Optional audio observations do not change completeness rules or their result.

6 3 Integrity

Integrity measures consistency among bound files, technical declarations, relationship parameters and independent cross-evidence. Malformed optional comparison values make only that comparison unavailable. Structural validity, C2PA attestation, legacy physical placeholders and optional audio observations do not contribute to Integrity. The axis does not authenticate the creator.

Component	Internal calculation	Evidence examined
File integrity	Passed checks divided by assessed Boolean checks; unavailable checks are excluded	SHA-256 binding, technical claims, and parser status.
Structural validity	Reported but not scored	Graph references, cycles, connectivity, final node, and endpoint roles.
Declaration consistency	Passed checks divided by all assessed checks	Submitted relationship ranges, duration, sample rate, bit depth, and other declared parameters.
Physical and audio observations	Not scored; no weight	Legacy physical placeholders stay disabled. Optional fingerprints and structure produce candidate observations only.
Cross-evidence support	Corroborated checks divided by assessed Boolean checks; malformed optional comparisons are unavailable	RIN, ERN, MIDI, sheets, embedded and standalone C2PA ingredients, and declared graph data.
Cryptographic attestation	Separate attestation axis; not scored in integrity	Only C2PA records bound to the final artefact affect the axis; supporting records remain visible.

The active policies/scoring_profile.json combines the available file_integrity, declaration_consistency and cross_evidence_support layers using positive configured weights, each 1.0 by default. Profile default-v2 tags every cross-evidence check with an evidence_tier: signed_attestation (validated C2PA), recorder_metadata (sound report or AAF compared with BWF bext/iXML written by the recorder into a hash-bound WAV) or submitter_document (RIN, ERN, CompSheet, CueSheet, MIDI, AAF, sound report or unverified C2PA alone). The default withholding rule requires at least two populated layers and an assessed check from an independent tier, unless an assessed contradiction permits a numeric result; submitter documents corroborate but cannot release a value on their own. Confidence is 0.8 multiplied by populated layers divided by 3. If an aggregated failed check matches severe_contradiction_rules, the value becomes min(weighted_mean, 0.25); the ceiling never raises a lower value. When the cap applies, the Integrity reasoning names the triggering rule(s) and the uncapped weighted mean, and assessment_checks.severe_contradiction_cap records the cap, triggered rules, weighted mean, final value and whether the value was lowered. The current severe rules cover object hash binding, RIN and ERN audio bindings, C2PA ingredient and standalone-manifest bindings, and CompSheet selection-to-edge consistency. Input errors that stop core execution return no score. Every successful response carries assessment_checks.score_explanations, one entry per axis, listing layers, weights, the weighted mean, whether a cap applied or a value was withheld, and every failed check as a sentence with its compared values (for example a 2.00 s gap between a sound report and the recorder timecode); a timecode mismatch is an ordinary, explained contradiction rather than a severe rule. Evidence the selected workflow does not describe is reported as inconclusive (WORKFLOW_EVIDENCE_OUTSIDE_POLICY) and does not change completeness. Ableton Live sessions (project/daw-session) are parsed and checked, as submitter documents, for the declared source audio, MIDI tempo and time signature. Audio candidates neither contribute to the mean nor trigger or cancel the cap. Real-evidence validation of these policy parameters remains future work.

6 4 Attestation Strength

Attestation strength reports the C2PA manifest, signature, SDK validation, and credential trust state bound to the final artefact. Supporting-node and standalone manifests are listed separately in c2pa_attestation_records and cannot raise the top-level axis. Embedded and standalone ingredient hashes and relationships are normalised and cross-checked against the submitted graph. The axis measures cryptographic provenance strength and does not establish that every semantic claim in a manifest is true.

Observed state	Axis result	Interpretation
SDK unavailable or no final-bound record	unavailable, null	Cryptographic attestation of the final artefact was not assessed; supporting records cannot raise the axis.
Reader failure	partial, null, confidence 0.3	Failure remains distinct from manifest absence.
No active manifest	available, 0.0, confidence 0.9	The reader confirmed no usable active manifest for the final artefact.
Invalid	available, 0.0, confidence 0.9	The SDK reported an invalid validation state for the final artefact.
Valid	available, 0.8, confidence 0.9	The final artefact is cryptographically valid; credential trust was not reported.
Trusted	available, 1.0, confidence 0.9	The final artefact is cryptographically valid and its credential is trusted.
Readable but unclear	partial, 0.5 or 0.25, confidence 0.6	0.5 with a signature and 0.25 without one.

6 5 AI Disclosure

AI disclosure measures explicit creation_method coverage across retained audio artefacts. The value is the number of audio nodes with a non-empty declaration divided by all retained audio nodes. If at least one declaration exists, confidence is 0.8. If none exists, the axis is unavailable.

A value of 1.0 means that every retained audio node has a declaration. It does not mean that the work contains no AI-generated material. A zero or null value does not prove AI generation.

6 6 Supporting Response Fields

Field	Content	Purpose
execution_status	succeeded, unsupported, error, timeout	Separates protocol execution from assessment results.
coverage	Submitted, evaluated, and skipped artefact and relationship counts	Prevents a partial graph from appearing fully assessed.
unknown_inputs	Original unknown value, kind, hashes, and skipped action	Reports unknown declarations without inference or correction.
validation	C2PA state, signature, trust, codes, and reasons	Preserves final-scoped cryptographic validation details.
assessment_checks	Core check arrays plus audio_fingerprint, audio_structure and audio_diagnostic_summary when enabled.	Separates evidence checks from diagnostic candidates, provider versions, parameters, ranges and limitations. Retains disabled physical placeholders.
workflow_assessment	Policy, workflow, modifiers, summary, and rule results	Explains the completeness result rule by rule.
findings	Core observations and errors, plus informational AUDIO_* findings with diagnostic_id and detail_key.	Links an optional observation to its measurements without making it a scored failure or an acceptance decision.
resource_usage	cpu_ms and peak_rss_kib	Records available resource information; peak RSS may be null.

7 Program File Responsibilities

The following inventory covers runtime code, schemas, policy, tests, examples, reports, and documentation. Content-addressed binaries and repeated public result files are grouped by directory. Cache files such as __pycache__, pyc, DS_Store, and Rhistory are not program logic.

7 1 Root Files

File	Responsibility
run.py	Main entry point and core worker. Local --audio-analysis flags enable a parent-side diagnostic stage after the validated core response; all work shares the caller timeout.
check_public.py	Runs a public corpus and writes per-case results plus public-summary.
make_examples.py	Generates deterministic WAV examples, hash failures, and unsupported media fixtures.
requirements.txt	Pinned required packages including jsonschema, numpy, pypdf, and xmlschema.
requirements-c2pa.txt	Optional c2pa-python dependency in addition to required packages.
README.md	Installation, execution, supported scope, quarantine behaviour, and current boundaries.
make_audio_diagnostic_example.py	Generates the repeated-source and four-second excerpt teaching fixture.
requirements-audio.txt and requirements-audio-lock.txt	Optional provider dependencies and the environment used for verification.

7 2 music_target

File	Responsibility
music_target/__init__.py	Declares core version 0.7.0 and the absence of an authorship classification policy.
music_target/contract.py	Request adaptation, input validation, quarantine and source identity. The source revision includes audio modules, dependency records and upstream licence/provenance files.
music_target/domain.py	Registers artefact and relationship types, endpoint contracts, and relationship attributes.
music_target/engine.py	Checks the final object first, hashes and binds files once, isolates non-final object problems, inspects WAV structure through wav_io (integer PCM and IEEE float, including WAVE_FORMAT_EXTENSIBLE), isolates non-final WAV parser failures, runs WavMetadataPass and the relationship EvidencePass, and orchestrates assessments.
music_target/assessment_passes.py	Aggregates cached file and relationship checks, isolates malformed optional comparisons, normalises standalone and embedded C2PA records, and runs final-scoped attestation, AI disclosure, and integrity combination logic.
music_target/audio_derivation.py	Deprecated signal utilities and exact-reconstruction code. Not executed or scored by the core assessment; the new runner does not reactivate them.
music_target/physical_audio_passes.py	Deprecated legacy physical-similarity passes, including the former eight diagnostics. Not invoked by the assessment or new providers.
music_target/workflow_policy.py	Loads policy safely, evaluates triggers, and reports workflow completeness; artefacts isolated by input errors count as not assessed rather than missing.

File	Responsibility
music_target/wav_parser.py	Extracts embedded WAV metadata (LIST/INFO, bext, cue, smpl, fact, ID3) chunk by chunk through wav_io, reading at most 1 MiB per metadata chunk.
music_target/wav_io.py	Streaming RIFF/WAVE inspection with chunk-safety checks and stable warnings, and vectorised decoding of integer PCM and IEEE float, including EXTENSIBLE, into float32 NumPy arrays.
music_target/midi_parser.py	Size- and structure-bounded MIDI parser.
music_target/sheet_parsers.py	CompSheet and CueSheet JSON, CSV, text, and PDF parsing.
music_target/scoring_profile.py	Validates three-layer weights, withholding and confidence settings, and severe rule/cap configuration.
music_target/audio_diagnostics/	Report-only contracts, local policy, task selection, snapshots, isolated workers, provider adapters and result assembly.

7 3 legacy_parsers

File	Responsibility
legacy_parsers/c2pa_parser.py	Optional SDK reader that distinguishes SDK, reader, manifest, and validation states.
legacy_parsers/rin_parser.py	RIN 2.1 contributors, sessions, components, and audio hash bindings.
legacy_parsers/ern_parser.py	ERN 4.3 releases, resources, parties, ISRC, and delivery audio bindings.
legacy_parsers/common/xml_utils.py	XML size bounds, DTD and entity rejection, and namespace helpers.
legacy_parsers/common/ddex_validation.py	RIN and ERN version registry and pinned local XSD validation.
legacy_parsers/common/__init__.py	Shared XML helper package marker.

7 4 Policies and Schemas

Path	Responsibility
policies/workflow_evidence_policy.json	Required, conditional, and optional expectations for 16 workflows.
schemas/native-request.schema.json	Native workflow, graph, object, and option contract.
schemas/scoring-request.schema.json	CSEC candidate request contract.
schemas/scoring-response.schema.json	Axes, coverage, unknown_inputs, validation, findings, checks, and execution status.
schemas/public-bundle.schema.json	Public CSEC bundle contract.
schemas/comp-sheet.schema.json	Normalised CompSheet JSON contract.
schemas/cue-sheet.schema.json	Normalised CueSheet JSON contract.
schemas/ddex/README.md	Pinned DDEX versions, offline use, and limitations.
schemas/ddex/SOURCES.json	DDEX source, version, and hash records.
schemas/ddex/rin-2.1	RIN 2.1 main, allowed-value, and XML signature schemas.

Path	Responsibility
schemas/ddex/ern-4.3	ERN 4.3 release-notification and allowed-value schemas.
policies/scoring_profile.json	Default equal weights for three Integrity layers, confidence ceiling 0.8 and severe cap 0.25.
policies/audio_analysis_policy.json	Provider selection and exploratory matching/feature parameters; no score or development-cap controls.
schemas/audio-diagnostic.schema.json	Result validation; scoring_eligible=false, integrity_contribution=null and content_match_is_provenance=false.

7 5 vendor libevchain

File or directory	Responsibility
artefact.py	Artefact nodes, attributes, file binding, Pass cache, and dependency call stack.
evidence.py	Evidence edges, relationship attributes, edge Pass cache, and cycle detection.
evidence_chain.py	Graph construction, reference completion, graph-cycle detection, and hash binding.
hash_methods.py	Hash method registry and SHA-256 implementation.
exceptions.py	Malformed JSON, attribute, Pass, and circular evaluation exceptions.
score.py	Upstream four-field Score container; the target uses axisAssessment instead.
pipeline/artefact_pass.py	ArtefactPass interface.
pipeline/evidence_pass.py	EvidencePass interface.
pipeline/pipeline.py	Pass registration, node and edge execution, and score merger invocation.
types/artefact_types.py	Artefact registration, inherited attributes, and Pass binding.
types/relationship_types.py	Relationship registration, generic specialisation, and Pass binding.
types/attribute.py	Any, Boolean, String, Number, List, Dictionary, Either, Literal, and Record validation.
types/artefact_audio_type.py	Base audio and speech types.
types/artefact_text_type.py	Base text type and text-reading helpers.
types/relationship_draft_of_type.py	Generic draft-of relationship registration.
types/__init__.py	Loads and exports built-in type and registration interfaces.
builtin_passes/c2pa_pass.py	Upstream C2PA draft; not used by the main target path.

7 6 Tests Examples Reports and Documentation

Path	Responsibility
tests/test_target.py	124 core tests covering inputs, graph safety, parsers, scoring profiles, severe caps, workflows, file isolation and legacy-pass non-invocation.
examples/valid-generated	Deterministic raw-track to stem to mix to master teaching example.

Path	Responsibility
examples/invalid-hash	Fixture whose bytes no longer match the declared SHA-256.
examples/unsupported-png	CSEC case with an unsupported final PNG.
examples/csec-wav	Single-WAV CSEC candidate request and response.
reports/verification.md	Historical core verification snapshot. Use audio-diagnostics-verification.json for the current integration test record.
reports/release.json	Historical release metadata, runtime and dependency snapshot; not the current optional-audio verification record.
reports/source_inventory.json	Initial source inventory and SHA-256 values.
reports/test-results.txt	Saved test output.
reports/public-summary.json	Execution-status summary for public cases.
reports/public-results	Per-case public request and response files.
docs/03_CSEC_HANDOFF.md	CSEC execution, exchange format, quarantine, and support boundaries.
docs/NEXT_TASKS.md	Prioritised roadmap; assess legacy signal proposals against the current policy-disabled boundary.
docs/SHEET_FORMATS.md	CompSheet and CueSheet fields and supported formats.
tests/test_audio_diagnostics.py	34 tests including the per-task worker, fingerprint agreement, CompSheet end derivation, real providers, repeated positions, a 3-minute long-source excerpt, length-independent SpectroMap peak density, time restoration, failure isolation and stderr reporting, snapshots and unchanged core results.
examples/audio-diagnostics/	Generated WAVs, request, core-response.json, response.json and a README explaining the repeated candidates.
reports/audio-diagnostics-verification.json	19 October 2026 record: 189 passed, no failures or skips; three real providers complete and core axes/findings unchanged.
docs/AUDIO_DIAGNOSTICS.md	Detailed optional installation, CLI usage, output fields, dependency provenance and interpretation.

8 Outstanding Work

The next priority is real-session validation of the three report-only providers, including repeated content, processing, misleading source direction and unrelated material. The 18 October 2026 integration passed 189 engineering tests and retained identical core results on the generated example. This does not establish real-recording accuracy. Matching thresholds need calibration before stronger diagnostic labels are introduced; existing scoring weights and the cap remain separate policy decisions.

Priority	Task	Required work	Completion criterion
P0	Real audio fixtures	Collect licensed source/sample, mix/master, edit and comp cases, plus unrelated, repeated and misleading-direction controls.	Reviewed time annotations and reproducible positive/negative cases.
P0	Diagnostic threshold calibration	Evaluate audfprint, SpectroMap and librosa by relation; measure false matches, misses, timing error and ambiguity.	Freeze parameters on calibration data and report held-out performance; retain report-only scope.

Priority	Task	Required work	Completion criterion
P0	Score policy validation	Review the three Integrity weights, withholding, confidence heuristic and severe cap independently of audio thresholds.	Document policy rationale and sensitivity; no automatic audio score contribution.
P0	Parser and graph reuse	Develop a shared request context for parsers and graph indexes.	No cross-request leakage or repeated expensive graph work.
P0	Pass dependency cycles	Extend explicit state and dependency-path diagnostics.	Tests cover cycles, diamond dependencies and exception cleanup.
P1	Submission CLI	Guide explicit workflow, roles, creation methods and directed relations.	Creation, validation and preview share the existing contract.
P1	C2PA and DDEX fixtures	Add valid and invalid real-standard examples with reviewed source/redistribution terms.	Offline positive and negative regressions cover scoped states and bindings.
P1	Workflow coverage	Exercise complete, missing, not-assessed and conflicting evidence in all workflows.	Every required and conditional rule has a reviewed case.
P1	Clean environment and profiling	Reproduce the dependency lock and profile real recordings with the optional providers, and run benchmarks/music_benchmark.py on BabySlakh, Slakh2100 or MUSDB18-HQ to calibrate thresholds with true and control stems.	All 189 tests pass; runtime and memory are measured without silently truncating files.
P1	Input-error policy review	Review isolated-input versus contradiction semantics as a separate scoring decision.	Any change is explicitly versioned and regression-tested.
P2	Additional audio formats	Prioritise real customer FLAC, AIFF, RF64/BW64 and codec needs.	Add format validation and binding before exposing them to providers.
P2	CrossEvidence and CSEC	Expand reviewed identifiers and agree workflow/non-WAV exchange extensions.	Every rule has traceable evidence and verified partner compatibility.
P3	Submission and report UI	Expose the existing contract and distinguish core scores from audio candidates.	Declarations stay explicit and diagnostic uncertainty is visible.
P3	DAW support	Prefer official exports or interchange data for proprietary projects.	No extension alone assigns a role or relationship.

8 1 Recommended Pass Reuse Dependencies

Cached producer	Downstream consumers
WavMetadataPass	RelationshipIntegrityPass, BoundFileIntegrityPass, and ExpandedCrossEvidencePass.
DeclaredTypeParserPass proposed	BoundFileIntegrityPass, ExpandedCrossEvidencePass, C2PAAttestationPass, and WorkflowCompletenessPass.
GraphIndexPass proposed	RelationshipIntegrityPass, ExpandedCrossEvidencePass, and WorkflowCompletenessPass.
Core WAV descriptors and diagnostic snapshots	Metadata checks reuse core descriptors; optional workers share verified temporary snapshots within one run. Old PCM/proxy helpers remain inactive in assessment.
Three scored Integrity layers	assess_axes applies configured weights, withholding, confidence and cap; optional audio results are never inputs.

Reuse must preserve declared semantics and evidence scope. Parsers, C2PA and DDEX can cross-check the submitted graph but cannot create relationships. Optional fingerprint and structure candidates remain outside all scored checks, use original-file time coordinates, and never overwrite ranges or gains. The deprecated reconstruction and physical passes stay outside the active path.